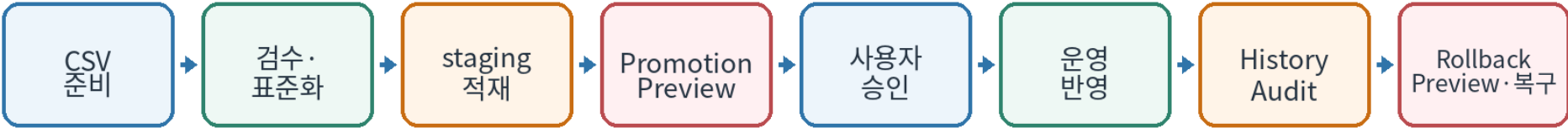


CatalogGuard Lite

패션·이커머스 상품 데이터 품질 검수 및 안전한 운영 반영 백엔드

한 줄 소개 서로 다른 공급사 CSV 를 JSON Profile 로 표준화·검수하고 PostgreSQL staging 에 적재한 뒤, Preview·승인·Audit·Rollback 으로 운영 상품 변경을 보호한 Python/FastAPI 개인 프로젝트입니다.

1923 passed 전체 pytest 회귀 기준	5 / 5 CI test · browser-e2e · K8s · Terraform · Airflow	2 Chromium Reject + Promotion/Rollback 실제 브라우저 E2E	v13 / v2 Inspection / ETL Profile 버전
--------------------------------	--	---	---



프로젝트에서 해결한 핵심 문제

문제	해결
공급사마다 다른 CSV 구조	JSON Profile + 공통 ETL Core 로 표준 스키마 변환
잘못된 상품의 운영 유입	규칙 기반 검수 후 normal / reject 분리, staging 선적재
운영 반영 실수	Preview → 사용자 승인 → Hash 재검증 → Transaction Promotion
반영 후 문제 발생	현재 상태 충돌 검사 후 Rollback, 실행/상품 단위 Audit 기록

GitHub psy0635-ctrl/catalogguard-lite · 기준일 2026-08-17

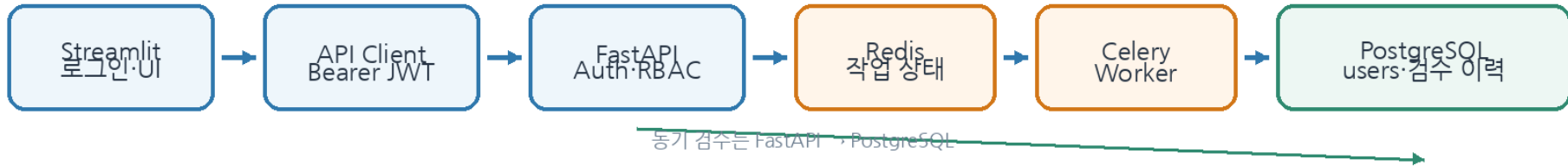
1. 시스템 구조와 설계 원칙

화면에서 DB 를 직접 수정하지 않고, 인증·API·Service/Query·DB 의 책임과 트랜잭션 경계를 분리했습니다.

CatalogGuard Lite 전체 구조

로그인·JWT·검수·ETL·Promotion·Rollback을 분리하고 PostgreSQL을 공통 저장 기준으로 사용

인증 + 동기·비동기 검수



공급사 ETL → staging → Promotion → 필요 시 Rollback



UI는 DB에 직접 쓰지 않고 CatalogGuardApiClient → FastAPI(Auth/RBAC) → Service/Query → PostgreSQL 흐름을 사용합니다.

계층별 책임

계층	역할	설계 판단
Streamlit	로그인, 검수/ETL/Promotion/Rollback UI	DB 직접 접근 없이 API Client 만 사용
FastAPI	Auth/RBAC, 검수, ETL, Promotion/Rollback API	JWT 와 현재 사용자 상태를 서버에서 최종 검증
Service / Query	쓰기 트랜잭션 / 읽기 전용 조회	transaction owner 를 Service 로 단일화
Redis · Celery	비동기 검수 작업 상태와 Worker	긴 검수를 HTTP 요청 수명과 분리
PostgreSQL	검수·staging·운영상품·Run·Audit 저장	constraint·unique·FK·transaction 으로 무결성 보장

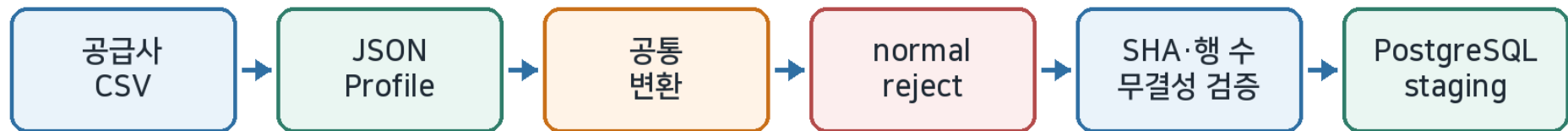
핵심 설계 원칙

- 단일 검수 기준: 검수 규칙은 FastAPI 서비스에서 한 번 실행하고 UI 는 저장된 결과를 표시해 이중 구현을 피했습니다.
- 권한 경계: UI 의 버튼 비활성화는 편의 기능이고, 실제 쓰기 권한은 FastAPI 가 viewer/operator RBAC 로 401/403 을 강제합니다.
- 읽기/쓰기 분리: 조회 전용 Query 와 쓰기 Service 를 나눠 N+1·pagination 과 transaction 책임을 분명히 했습니다.
- 공통 Core 재사용: Web/CLI/S3/configured HTTP feed 가 같은 ETL 변환·적재 로직을 재사용합니다.

왜 이 구조인가? 기능을 많이 붙이기보다 “검수되지 않은 데이터가 운영 DB 를 직접 바꾸지 않게 한다”는 안전 경계를 코드 구조와 DB 트랜잭션으로 유지하는 것을 우선했습니다.

2. 상품 데이터 품질 검수와 공통 ETL

패션 도메인 규칙과 공급사별 구조 차이를 하나의 표준화·검수 흐름으로 연결했습니다.



검수 범위

분류	검수 내용	예시
입력·형식	필수값, category·stock·price·sale_price	price="가격문의" → 오류
중복	상품 ID, 상품명 후보, 완전 중복	동일 ID/동일 속성 중복
패션 옵션	색상·사이즈 표준화, variant 중복	블랙→BLACK, medium→M
그룹 관계	product_group_id 내부 카테고리 일관성	TOP 과 BOTTOM 이 같은 그룹
가격	0 이하, 정상가/할인가 관계, 카테고리 이상치	sale_price > price
내용·보안	상품명-카테고리, 금지어, 개인정보 의심	설명에 이메일·전화번호

공급사 Profile 과 staging 안전장치

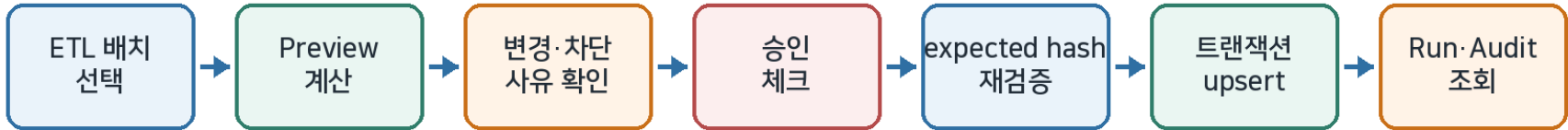
- 공급사별 컬럼명은 JSON Profile 로 표준 필드에 매핑해 변환 코드를 공급사마다 복제하지 않았습니다.
- 정상 행만 PostgreSQL staging 에 적재하고 reject·summary·SHA-256 을 함께 관리합니다.
- 동일 input hash + profile + version 은 기존 batch 를 재사용해 중복 적재를 방지합니다.
- Source Lineage 는 upload / s3 / http_feed / cli / unknown 을 기록하되 URL token·bucket 같은 민감한 원문은 저장하지 않습니다.

패션 size 정책 TOP/BOTTOM/OUTER/SHOES 는 size 필수, BAG 은 선택입니다. BAG 의 빈 값과 FREE 는 모두 허용하지만 서로 다른 값으로 유지합니다. 사전에 없는 size 는 그 값만으로 오류/주의를 만들지 않으며 fuzzy matching 을 사용하지 않습니다.

3. Promotion 과 Rollback - 운영 변경 보호

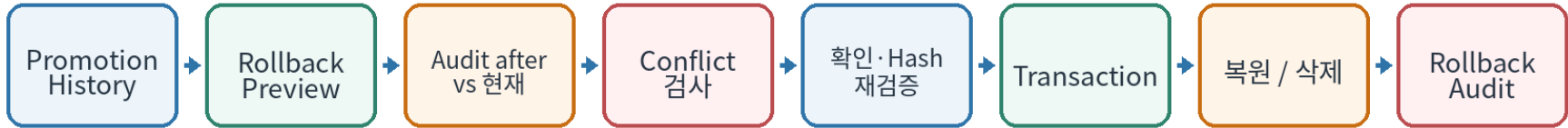
staging 데이터를 바로 운영에 반영하지 않고 미리보기·명시적 승인·Hash 재검증·트랜잭션·Audit 을 거치도록 설계했습니다.

Promotion: 반영 전에 “무엇이 바뀌는지” 확인



위험	안전장치
변경 내용을 모른 채 반영	insert/update/no-change Preview 와 상품별 변경 전·후 표시
Preview 후 데이터 변경	expected_preview_hash 를 실행 직전 재계산, 불일치 시 409 preview_stale
실수로 즉시 반영	operator RBAC + confirmed + UI 승인 checkbox
일부만 반영	단일 PostgreSQL transaction 으로 All-or-Nothing

Rollback: 최신 변경을 덮어쓰지 않는 복구



위험	안전장치
다른 최신 변경을 덮어쓰	원본 Audit after 와 현재 business field 를 비교해 conflict 차단
Rollback Preview 후 변경	rollback preview hash 를 실행 직전 재검증
중복 Rollback	FOR UPDATE + partial unique index 로 성공 Rollback 1 회 제한
복구 중 일부 실패	하나의 transaction 에서 삭제/복원, 실패 시 전체 원복

Audit Inspection·ETL·Promotion·Rollback Run 에 실행 주체를 기록하고, Promotion/Rollback 은 상품 단위 before/after 이력까지 남겨 “누가 무엇을 바꿨는지” 추적할 수 있게 했습니다.

4. 문제 해결 경험 - 테스트가 놓친 실제 DB 오류

“테스트가 많이 통과한다”보다 실제 실행 경로에서 실패를 재현하고 원인을 구조적으로 제거하는 데 집중했습니다.

대표 사례 전체 테스트가 통과하던 상태에서도 실제 PostgreSQL 동기 검수 Route 에서 HTTP 500 이 발생했습니다. 원인은 사전 SELECT 가 SQLAlchemy autobegin transaction 을 열고, 저장 Service 가 같은 Session 에서 session.begin()을 다시 호출한 transaction ownership 충돌이었습니다.

오류 분석과 수정 과정

단계	확인 내용	조치
1. 재현	실제 PostgreSQL Route 호출에서만 500	SQLite/단위 테스트가 아닌 route-level regression 추가
2. 원인	precheck SELECT → autobegin, 이후 Service session.begin() 충돌	세션의 트랜잭션 소유권을 추적
3. 수정	사전 조회와 저장이 같은 Session 을 공유	precheck_session 을 별도로 분리
4. 결과	Service 가 유일한 transaction owner 가 됨	중복 사전 조회 + DB unique 방어를 모두 유지

같은 방식으로 해결한 UI 회귀

Web ETL selectbox 가 기존 검색 필드와 같은 accessible label 을 사용해 Playwright strict-mode violation 이 발생했습니다. selector 를 .first()로 우회하지 않고 신규 label 을 “ETL 실행 프로필”로 분리하여 UI 의미와 E2E 안정성을 함께 회복했습니다.

성능 판단도 “근거가 있을 때만”

항목	기준
데이터 규모	inspection_runs 10,000 / inspection_results 100,000
측정	EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON), warmup 2 회 + 측정 7 회
중앙값 예시	동일 CSV 0.018 ms · 목록 0.044 ms · count 0.872 ms · 상세 0.016 ms
판단	현재 측정 범위에서 병목 근거가 없어 불필요한 index/migration 을 추가하지 않음

배운 점 테스트 개수나 기술 이름보다 “실제 장애를 재현할 수 있는가, 트랜잭션·DB 제약·브라우저 행동까지 검증하는가”가 백엔드 품질에 더 중요하다고 판단했습니다.

5. 테스트·CI와 운영 환경 검증

일반 테스트와 브라우저·인프라 검증을 분리하여 실패 원인을 빠르게 구분하고, 실제 서비스 연결 경로를 회귀 테스트로 남겼습니다.

1923 passed	2 Chromium E2E	5 CI jobs	0 failed
----------------	-------------------	--------------	-------------

GitHub Actions 5 개 검증 축

Job	검증 범위
test	PostgreSQL·Redis, Auth/RBAC, Sync/Async inspection, Web ETL, Promotion/Rollback, Streamlit, AWS runtime smoke
browser-e2e	Chromium 에서 Reject masking + Promotion→Audit→Rollback→Rollback Change Audit, PostgreSQL 최종 상태
kubernetes-smoke	kind 에서 PostgreSQL → Migration Job → API rollout, /health·/ready 확인
terraform-validate	EC2/RDS/SG/SSM 구조 validate + mock provider test, 실제 AWS apply 는 하지 않음
airflow-smoke	Airflow 3.3.0 configured HTTP feed manual DAG import/parse 및 기존 ETL 재사용 검증

실제 운영 환경과 범위 구분

영역	검증한 것	과장하지 않는 범위
Docker/AWS runtime	Dockerfile.aws build, UID 10001, migration, Uvicorn /health·/ready	최신 코드의 실제 EC2/RDS 재배포는 미실행
AWS staging 경험	EC2·RDS·TLS·Docker·FastAPI·Streamlit 검증 경험	비용 절감으로 리소스 중지
S3 ingestion	Instance Role 최소 권한으로 private S3 입력을 기존 ETL→staging 에 연결	정적 access key 미주입
Observability	Prometheus /metrics 및 HTTP/ETL 지표	대규모 운영 모니터링/SLA 검증은 아님

현재 검증 기준 2026-08-17 · main HEAD ffa20af... · GitHub Actions Run 31945356223 · 5/5 success · 1923 passed · 2 skipped · 5 deselected · 0 failed.
--

6. 지원 포지셔닝과 현재 한계

핵심은 인프라 기술의 개수가 아니라 패션·이커머스 상품 데이터를 안전하게 검수·적재·반영하는 백엔드 문제를 해결한 경험입니다.

핵심 기술 스택

Python	FastAPI	PostgreSQL	SQLAlchemy	Alembic	Redis	Celery	ETL
Streamlit	Playwright	Docker	GitHub Actions	AWS	Kubernetes(kind)	Terraform	Airflow

채용 관점에서 보여주고 싶은 역량

역량	프로젝트 근거
Python/FastAPI 백엔드	API 계약, JWT/RBAC, Service/Query 분리, 상태 코드·예외 처리
DB·트랜잭션	PostgreSQL constraint/unique/FK, Promotion/Rollback 원자성, transaction ownership 수정
상품 데이터·ETL	JSON Profile, normal/reject, staging, SHA-256 dedup, Source Lineage
테스트·운영 품질	실제 PostgreSQL regression, Chromium E2E, CI 분리, runtime smoke

현재 한계 - 구현하지 않은 것을 명확히 구분

- Refresh Token·회원가입/password reset·OAuth/MFA/SSO·rate limit·세밀한 ACL/관리자 화면은 현재 MVP 범위 밖입니다.
- Web 업로드는 동기·단일 CSV 중심이며 streaming·증분 ETL·대규모 동시 트래픽 부하 시험은 지원하지 않습니다.
- Airflow 는 configured HTTP feed 의 manual trigger 범위이고, Terraform CI 는 실제 AWS apply 를 하지 않습니다.
- Kubernetes 는 kind smoke 기준이며 production cluster·autoscaling·장애 복구 운영까지 검증한 것은 아닙니다.
- 성능 수치는 제한된 합성 benchmark 이며 운영 트래픽 성능이나 SLA 를 보증하지 않습니다.

30초 소개 CatalogGuard Lite 는 공급사 상품 CSV 를 JSON Profile 로 표준화하고 품질 오류를 검수하는 Python·FastAPI 백엔드 프로젝트입니다. 정상 데이터만 PostgreSQL staging 에 적재하고, Preview Hash 기반 Promotion 과 conflict-safe Rollback 으로 운영 변경을 보호합니다. 실제 PostgreSQL 회귀 테스트와 Chromium E2E, 분리된 CI 를 통해 기능뿐 아니라 데이터 무결성과 운영 안전성까지 검증했습니다.

상세 기술 문서는 별도 13 페이지 버전으로 보관 · 이 문서는 채용담당자 1 차 검토용 요약본